



Nội dung

- Trong khóa Đồng, bạn đã học BFS/DFS cơ bản trên đồ thị. Bài này tập trung vào ứng dụng phổ biến nhất: **lưới (grid)** — tìm thành phần liên thông, đường đi ngắn nhất, lan tỏa từ nhiều nguồn.

Khai báo dx, dy

Python

$$\begin{aligned} DX &= [-1, 0, 1, 0] \\ DY &= [0, 1, 0, -1] \end{aligned}$$



BFS lưới

C++

Python

```
bool vis[N][N];
char g[N][N];

void bfs(int sx, int sy) {
    queue<pair<int,int>> q;
    q.push({sx, sy});
    vis[sx][sy] = true;
    while (!q.empty()) {
        auto [x, y] = q.front(); q.pop();
        for (int d = 0; d < 4; d++) {
            int nx = x + dx[d], ny = y + dy[d];
            if (nx < 1 || nx > n || ny < 1 || ny > m) continue;
            if (g[nx][ny] == '#' || vis[nx][ny]) continue;
            vis[nx][ny] = true;
            q.push({nx, ny});
        }
    }
}

int islands = 0;
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++)
        if (g[i][j] == '.' && !vis[i][j]) { bfs(i, j); islands++; }
```



```
DX, DY = [-1, 0, 1, 0], [0, 1, 0, -1]
vis = [[False] * (m + 1) for _ in range(n + 1)]

def bfs(sx, sy):
    q = deque([(sx, sy)])
    vis[sx][sy] = True
    while q:
        x, y = q.popleft()
        for d in range(4):
            nx, ny = x + DX[d], y + DY[d]
            if not (1 <= nx <= n and 1 <= ny <= m): continue
            if g[nx][ny] == '#' or vis[nx][ny]: continue
            vis[nx][ny] = True
            q.append((nx, ny))

islands = 0
for i in range(1, n + 1):
    for j in range(1, m + 1):
        if g[i][j] == '.' and not vis[i][j]:
            bfs(i, j)
            islands += 1
```

Lưu ý: trên lưới $n \times m$ lớn (ví dụ 1000×1000), DFS đệ quy có thể **stack overflow** — nên dùng **BFS** (vòng lặp với queue).

2. BFS = Đường đi ngắn nhất trên đồ thị không trọng số

Trên lưới với mọi bước có **trọng số 1**, BFS từ một đỉnh cho khoảng cách ngắn nhất tới mọi đỉnh khác.



Tiếng Việt



		4			4		
	4	3	3	3	3	4	
4	3	2	2	2	2	3	4
4	3	2	1	1	2	3	4
	1	1	0	1	1	2	
4	3	2	1	1	2	3	4
4	3	3	2	3	3	3	4
	4	4		4	4	4	

0 = nguồn

1, 2, 3, 4 = số cạnh
tới nguồn

BFS lan tỏa từ nguồn — mỗi vòng lặp thêm 1 lớp

Hãy hình dung BFS như **sóng lan ra từ nguồn**: vòng lặp đầu tiên chạm các ô cách 1 cạnh, vòng tiếp theo chạm các ô cách 2 cạnh, và cứ thế. Khi một ô được chạm lần đầu tiên, đó **chắc chắn** là khoảng cách ngắn nhất tới ô đó.

C++

Python



```

fill(&dist[0][0], &dist[0][0] + N*N, -1);
queue<pair<int,int>> q;
q.push({sx, sy});
dist[sx][sy] = 0;
while (!q.empty()) {
    auto [x, y] = q.front(); q.pop();
    for (int d = 0; d < 4; d++) {
        int nx = x + dx[d], ny = y + dy[d];
        if (nx < 1 || nx > n || ny < 1 || ny > m) continue; // ngoài lưới
        if (g[nx][ny] == '#') continue; // tường
        if (dist[nx][ny] != -1) continue; // đã thăm
        dist[nx][ny] = dist[x][y] + 1;
        q.push({nx, ny});
    }
}

```

```

from collections import deque
dist = [[-1] * (m + 1) for _ in range(n + 1)]
q = deque([(sx, sy)])
dist[sx][sy] = 0
while q:
    x, y = q.popleft()
    for d in range(4):
        nx, ny = x + DX[d], y + DY[d]
        if not in_bounds(nx, ny) or g[nx][ny] == '#':
            continue
        if dist[nx][ny] != -1:
            continue
        dist[nx][ny] = dist[x][y] + 1
        q.append((nx, ny))

```

3. Multi-source BFS



Tiếng Việt



ô (x, y) bị lửa đèn?

Ý tưởng: Push **tất cả** nguồn vào queue **cùng lúc** với khoảng cách 0. BFS bình thường — `dist[x][y]` sẽ là thời gian ô đó bị lửa.

C++

Python

```
queue<pair<int,int>> q;
for (auto [sx, sy] : sources) {
    q.push({sx, sy});
    dist[sx][sy] = 0;
}
// BFS như thường
```

```
from collections import deque
q = deque()
for sx, sy in sources:
    q.append((sx, sy))
    dist[sx][sy] = 0
# BFS như thường
```

Tại sao đúng? Tưởng tượng có một "siêu nguồn" nối đến mọi nguồn với cạnh 0. BFS từ siêu nguồn = BFS multi-source.

4. 0-1 BFS — khi đi qua ô có giá khác nhau

Đề mẫu: Lưới $n \times m$. Mỗi ô là `.` (đi qua **miễn phí**, chi phí 0) hoặc `#` (đi qua tốn **1 đơn vị**, ví dụ "đẩy tường"). Tìm chi phí nhỏ nhất từ (sx, sy) đến (tx, ty) .

Vấn đề: BFS thường giả sử mọi bước có giá bằng nhau, nên không xử lý được trường hợp này. Còn Dijkstra (sẽ học bài 11) tổng quát hơn nhưng có hệ số $\log$.

Mẹo 0-1 BFS: Khi giá chỉ có 2 mức là **0 và 1**, dùng **deque** thay **queue**:

- Bước "miễn phí" (giá 0) → đẩy vào **đầu** deque (`push_front`).
- Bước "tốn 1" (giá 1) → đẩy vào **đuôi** deque (`push_back`).
- Vẫn lấy ra từ đầu như BFS thường.



BFS với "khoảng cách" mở rộng. Phức tạp vẫn là $O(o)$, không có log.

C++ Python

```
int dist[N][N];
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++) dist[i][j] = INT_MAX;

deque<pair<int,int>> dq;
dq.push_front({sx, sy});
dist[sx][sy] = 0;

while (!dq.empty()) {
    auto [x, y] = dq.front(); dq.pop_front();
    for (int d = 0; d < 4; d++) {
        int nx = x + dx[d], ny = y + dy[d];
        if (nx < 1 || nx > n || ny < 1 || ny > m) continue;
        int w = (g[nx][ny] == '#') ? 1 : 0;      // Giá đi vào ô (nx, ny)
        if (dist[x][y] + w < dist[nx][ny]) {
            dist[nx][ny] = dist[x][y] + w;
            if (w == 0) dq.push_front({nx, ny});
            else        dq.push_back({nx, ny});
        }
    }
}
```



```

INF = float('inf')
dist = [[INF] * (m + 1) for _ in range(n + 1)]
dist[sx][sy] = 0
dq = deque([(sx, sy)])

while dq:
    x, y = dq.popleft()
    for d in range(4):
        nx, ny = x + DX[d], y + DY[d]
        if not (1 <= nx <= n and 1 <= ny <= m): continue
        w = 1 if g[nx][ny] == '#' else 0
        if dist[x][y] + w < dist[nx][ny]:
            dist[nx][ny] = dist[x][y] + w
            if w == 0:
                dq.appendleft((nx, ny))
            else:
                dq.append((nx, ny))

```

Lưu ý: code trên dùng `dist[x][y]` hiện tại (không phải giá trị `d` lưu lúc push) khi relax — vì vậy **không cần kiểm tra "stale"** như ở Dijkstra. An toàn và đúng.

“

Mở rộng: Khi giá có **nhiều giá trị** (> 2), 0-1 BFS không đủ — cần Dijkstra (bài 11).

5. BFS với trạng thái mở rộng

Đôi khi đỉnh không chỉ là (x, y) mà còn kèm trạng thái: "có chìa khóa hay không", "đang quay hướng nào", v.v.

Mẹo: encode trạng thái thành đỉnh $(x, y, state)$. BFS trên đồ thị mở rộng.

C++ Python

```
int dist[N][N][K]; // K = số trạng thái; đỉnh = (x, y, state)
```



Tiếng Việt



```
# đỉnh = (x, y, state)
```

6. DFS trên lưới — đếm kích thước vùng

C++

Python

```
int dfs(int x, int y) {
    vis[x][y] = true;
    int sz = 1;
    for (int d = 0; d < 4; d++) {
        int nx = x + dx[d], ny = y + dy[d];
        if (nx < 1 || nx > n || ny < 1 || ny > m) continue;
        if (g[nx][ny] == '#' || vis[nx][ny]) continue;
        sz += dfs(nx, ny);
    }
    return sz;
}
```

```
def dfs(x, y):
    vis[x][y] = True
    sz = 1
    for d in range(4):
        nx, ny = x + DX[d], y + DY[d]
        if hop_le(nx, ny):
            sz += dfs(nx, ny)
    return sz
```

Trên lưới $\leq 1000 \times 1000$, DFS đệ quy có thể stack overflow nếu vùng kết nối lớn → dùng BFS.

7. Mẫu bài toán hay gặp

- **Đếm đảo** (số vùng đất liền thông).



Tiếng Việt



- **Lưu lan tỏa, người thoát** (multi-source BFS + so sánh).
- **Mê cung có chìa khóa** (BFS với trạng thái).
- **Đếm số ô có thể đi tới** từ vị trí cho trước.

8. Bẫy thường gặp

- **Đánh dấu** `vis` **ngay khi push** vào queue (không phải khi pop) — nếu không sẽ push trùng → TLE.
- **Boundary**: kiểm tra ngoài lưới trước khi đọc `g[nx][ny]`.
- **DFS đệ quy quá sâu** trên lưới lớn → BFS.
- **Quên reset** `vis` / `dist` giữa các test case.

9. Tổng kết

BFS/DFS trên lưới là **một trong những dạng bài phổ biến nhất** ở mức Bạc. Hãy luyện đến mức:

- Code BFS lưới trong 2 phút mà không nghĩ.
- Tự nhận ra bài "lan tỏa" → multi-source BFS.
- Biết khi nào dùng BFS vs DFS (trên lưới lớn → BFS).

Sau bài này, bạn nên có thể giải hầu hết các bài "grid + connectivity" trên các Online Judge mức Silver!

Bài tập

	Doraemon tự kỷ với trò chơi mới	0 / 10
	KNIGHT	0 / 15
	Bảng số	0 / 15
	Tìm đường đi ngắn nhất trong mê cung	0 / 15
	Ao làng	0 / 20
	KING	0 / 20
	Sơ tán	0 / 20



Tiếng Việt



 Biến đổi về Zero	0 / 25
 Đẩy hộp	0 / 25
 Hành tinh đầm lầy	0 / 25
 CSES - Monsters Quái vật	0 / 30

proudly powered by **DMOJ** | [developed by LQDJudge team](#) Tiếng Việt (vi) ▼